

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

Съдържание:

1. Pattern matching

- 1.1. Присвояване
- 1.2. Съвпадение
- 1.3. Деструктуриране
- 1.4. Съвпадащ модел
- 1.5. Match operator
- 1.6. Matching tuples
- 1.7. Matching constants
- 1.8. Variables in patterns
- 1.9. Matching lists
- 1.10. Matching maps
- 1.11. Matching bitstrings and binaries
- 1.12. Matching binary strings
- 1.13. Compound matches
- 1.14. General behavior
- 1.15. Matching with functions
- 1.16. Multiclaue functions

1.17. Guards

1.18. Многокузни ламбди

2. Performing Pattern Matching

- 2.1. Търся модели в данните
- 2.2. Разбиране на регулярните изрази
- 2.3. Дефиниране на специални символи с помощта на бягства
- 2.4. Определяне на заместващи знаци
- 2.5. Работа с anchors (котви)
- 2.6. Очертаване на подвидове с помощта на групиращи конструкции
- 2.7. Използване на съвпадение на шаблони в анализа
- 2.8. Използване на двигателя за съвпадение на шаблони на виртуалната машина
- 2.9. Match patterns
 - 2.9.1. Операторът за съвпадение
 - 2.9.2. Съвпадение на шаблони
 - 2.9.3. pin операторът

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

В Elixir, контролът на пътя на изпълнение на програмата с помощта на "pattern matching" (проверка на модели) е един от основните концептуални механизми, който позволява на разработчиците да оформят логиката на приложенията си по изключително ефективен начин.

Тук са разгледани принципите и правилата за прилагане на "pattern matching" за различни типове данни, както и неговото приложение при дефиниране на функции.

1. Какво е Pattern Matching?

"Pattern matching" в Elixir представлява техника за разпознаване на структурата на данните и извличане на техните стойности, когато данните отговарят на зададен шаблон. Вместо да сравнявате стойности, в Elixir можете директно да разпознаете структури от данни и да присвоите стойности на променливи.

Пример:

```
{a, b} = {1, 2}
IO.inspect(a) # Изход: 1
IO.inspect(b) # Изход: 2
```

В този пример, стойностите 1 и 2 от кортежа {1, 2} автоматично се присвояват на променливите a и b.

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

2. Основни принципи на Pattern Matching

а. Структурираност

"Pattern matching" работи чрез сравняване на структури с определен вид. В зависимост от типа на данните (кортежи, списъци, мапове и т.н.), Elixir определя дали съвпадението е успешно.

б. Съвпадения

Когато две структури са равни (например, кортежи от еднаква форма), тяхното "pattern matching" е успешен. В противен случай, ще се получи грешка.

Успех

`{1, 2} = {1, 2} # Успешно`

Провал

`{1, 2} = {2, 1} # Грешка: no match of right hand side value`

с. Възможност за използване на символи за „грубо“ съвпадение

В Elixir можете да използвате `_` (подчертаване) за игнориране на определени стойности.

`{_, b} = {1, 2}`

`IO.inspect(b) # Изход: 2`

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

3. Прилагане на Pattern Matching към различни типове данни

а. Списъци

При "pattern matching" на списъци можете да извлечете елементи по индекс.

```
[head | tail] = [1, 2, 3]
```

```
IO.inspect(head) # Изход: 1
```

```
IO.inspect(tail) # Изход: [2, 3]
```

б. Кортежи

Можете да извлечете данни по структура:

```
{status, message} = {:ok, "Operation completed."}
```

```
IO.inspect(status) # Изход: :ok
```

с. Мапове

При работа с мапове можете да извлечете стойности по ключ:

```
map = %{name: "Elixir", type: "Programming Language"}
```

```
%{name: language_name} = map
```

```
IO.inspect(language_name) # Изход: "Elixir"
```

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

4. Приложение на Pattern Matching при дефиниране на функции

Реализации на функции, основани на параметрите, които получават.

Пример:

```
defmodule Math do
  def add({:ok, a, b}) do
    a + b
  end
```

```
  def add({:error, _reason}) do
    {:error, "Invalid parameters"}
  end
end
```

```
IO.inspect(Math.add({:ok, 1, 2}))    # Изход: 3
```

```
IO.inspect(Math.add({:error, "some error"})) # Изход: {:error, "Invalid parameters"}
```

В този пример, функцията `add/1` дефинира две различни поведения на базата на входния параметър. Първият вариант се активира, когато входът е кортеж с `:ok`, а вторият - когато е с `:error`.

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

Или може да обобщим

Пример 1: Присвояване на стойности от кортеж

```
{a, b} = {1, 2}
```

```
IO.inspect(a) # Изход: 1
```

```
IO.inspect(b) # Изход: 2
```

Пример 2: Игнориране на стойност

```
{_, b} = {1, 2}
```

```
IO.inspect(b) # Изход: 2
```

Пример 3: Списъци

```
[first | rest] = [1, 2, 3]
```

```
IO.inspect(first) # Изход: 1
```

```
IO.inspect(rest) # Изход: [2, 3]
```

Пример 4: Определяне на функция с pattern matching

```
defmodule Example do
```

```
  def greet(:morning), do: "Good morning!"
```

```
  def greet(:evening), do: "Good evening!"
```

```
end
```

```
IO.inspect(Example.greet(:morning)) # Изход: "Good morning!"
```

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

Пример 5: Кортежи и игнориране на стойност

```
{status, _msg} = {:ok, "Everything is fine"}  
IO.inspect(status) # Изход: :ok
```

Пример 6: Извършване на проверки

```
defmodule Check do  
  def check({:ok, value}), do: "Value is #{value}"  
  def check({:error, reason}), do: "Error: #{reason}"  
end  
IO.inspect(Check.check({:ok, 42})) # Изход: "Value is 42"  
IO.inspect(Check.check({:error, "fail"})) # Изход: "Error: fail"
```

Пример 7: Мапове

```
map = %{name: "Elixir", age: 10}  
%{name: name} = map  
IO.inspect(name) # Изход: "Elixir"
```

Пример 8: Непълен мап

```
map = %{name: "Elixir"}  
%{name: name, age: _} = map # age се игнорира  
IO.inspect(name) # Изход: "Elixir"
```

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

Пример 9: Групиране на данни

```
{first, second, third} = {1, 2, 3}
```

```
IO.inspect(first + second + third) # Изход: 6
```

Пример 10: Работа с nested кортежи

```
{{x, y}, z} = {{1, 2}, 3}
```

```
IO.inspect(x) # Изход: 1
```

```
IO.inspect(y) # Изход: 2
```

```
IO.inspect(z) # Изход: 3
```

Пример 11: Списъци с различни дължини

```
list = [1, 2, 3, 4, 5]
```

```
[a, b | _] = list
```

```
IO.inspect(a) # Изход: 1
```

```
IO.inspect(b) # Изход: 2
```

Пример 12: Определяне на множество функции с различни параметри

```
defmodule Math do
```

```
  def calculate({:add, a, b}), do: a + b
```

```
  def calculate({:subtract, a, b}), do: a - b
```

```
end
```

```
IO.inspect(Math.calculate({:add, 5, 3})) # Изход: 8
```

```
IO.inspect(Math.calculate({:subtract, 5, 3})) # Изход: 2
```



Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

Пример 13: Дефиниция на функция с по-сложен шаблон

```
defmodule Colors do
  def describe({:red, intensity}), do: "Red with intensity #{intensity}"
  def describe({:blue, intensity}), do: "Blue with intensity #{intensity}"
end
IO.inspect(Colors.describe({:red, 5})) # Изход: "Red with intensity 5"
```

Пример 14: Извличане на елемент от списък

```
list = [1, 2, 3]
[head | tail] = list
IO.inspect(head) # Изход: 1
IO.inspect(tail) # Изход: [2, 3]
```

Пример 15: Съответствие на стойности в мап

```
person = %{name: "Alice", age: 30}
%{name: name} = person
IO.inspect(name) # Изход: "Alice"
```

Пример 16: Скриване на стойност

```
{_, result} = {5, 10}
IO.inspect(result) # Изход: 10
```

Пример 17: Проверка на тип с pattern matching

```
defmodule TypeCheck do
  def type(:integer), do: "It's an integer"
  def type(:float), do: "It's a float"
  def type(:atom), do: "It's an atom"
end
IO.inspect(TypeCheck.type(:integer)) # Изход: "It's an integer"
```

Пример 18: Деконструиране на кортеж

```
{first, middle, last} = {1, 2, 3}
middle_plus_one = middle + 1
IO.inspect(middle_plus_one) # Изход: 3
```

Пример 19: Структури с pattern matching

```
defmodule User do
  defstruct name: "", age: 0
end
%User{name: user_name} = %User{name: "John", age: 30}
IO.inspect(user_name) # Изход: "John"
```

Пример 20: Сложна структура

```
defmodule Complex do
  def handle({:error, reason}), do: "Failed: #{reason}"
  def handle({:ok, %{name: name}}), do: "Success for #{name}"
end
IO.inspect(Complex.handle({:ok, %{name: "Alice"}})) # Изход: "Success for Alice"
```



Анализ към пример 19

Определяне на модул и структура:

`defmodule User do defstruct name: "", age: 0 end` - Тук дефинираме модул `User`, който съдържа структура `defstruct`. `User` структурата има два полета:

- `name`, което по подразбиране е зададено на празен стринг (`""`)
- `age`, което по подразбиране е зададено на `0`.

Тази структура представлява концепцията за потребител (`User`) с атрибутите име и възраст.

Pattern Matching на структура:

`%User{name: user_name} = %User{name: "John", age: 30}` – Тук се използва **pattern matching**, за да извлечем стойността на полето `name` от новосъздадена инстанция на структура `User`.

- `%User{name: user_name}` е шаблон, който „съвпада“ със структура `User` и извлича стойността на полето `name`, присвоявайки я на променливата `user_name`.
- `%User{name: "John", age: 30}` е самата структура, от която извличаме данните.

В този случай, `user_name` ще бъде присвоено стойността `"John"` от структурата.

Изход:

`IO.inspect(user_name) # Изход: "John",`

Тук просто извеждаме стойността на `user_name` на конзолата с помощта на `IO.inspect`. Тъй като `user_name` беше зададено да бъде `"John"`, това ще изведе `"John"` в конзолата.

Обобщение на основните стъпки:

1. Дефиниране на модул `User` с два полета: `name` и `age`.
2. Създаване на инстанция на структура `User` с конкретни стойности.
3. Използване на **pattern matching**, за извличане на името на потребителя от структурата.
4. Извеждане на името `user_name` на конзолата, което е `"John"`.

Този подход е често използван в `Elixir` за работа с структури и позволява лесно извличане на данни от тях, което прави кода много удобен и четим

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

5. Работата с регулярни изрази в Elixir е лесна и се осъществява чрез вградения модул Regex. С него можете да търсите модели в низове, да ги заменят и да извличат информация. По-долу са представени основните операции, свързани с регулярни изрази.

Основни операции с регулярни изрази в Elixir

- **Създаване на регулярни изрази**

Можете да създадете регулярни изрази, като използвате конструкцията `/pattern/`:

```
regex = ~r/\d+/ # Регулярен израз за търсене на цифри
```

- **Търсене на съвпадения**

Използвайте `Regex.match?/2`, за да проверите дали низът съвпада с регулярния израз:

```
regex = ~r/\d+/
```

```
match = Regex.match?(regex, "abc123")
```

```
IO.puts("Съвпадение: #{match}") # Изход: Съвпадение: true
```

- **Намиране на съвпадения**

Можете да използвате `Regex.run/2`, за да извлечете първото съвпадение:

```
regex = ~r/\d+/
```

```
match = Regex.run(regex, "abc123")
```

```
IO.inspect(match) # Изход: ["123"]
```

- **Намиране на всички съвпадения**

С `Regex.scan/2` можете да извлечете всички съвпадения:

```
regex = ~r/\d+/
```

```
matches = Regex.scan(regex, "abc123 def456")
```

```
IO.inspect(matches) # Изход: [["123"], ["456"]]
```

Замяна на съвпадения

Използвайте `Regex.replace/4`, за да замените съвпадения с друг низ:

```
regex = ~r/\d+/
```

```
result = Regex.replace(regex, "abc123 def456", "[NUMBER]")
```

```
IO.puts(result) # Изход: abc[NUMBER] def[NUMBER]
```

Пример за работа с регулярни изрази

Проверка на низ за имейл адреси и извличане на всички валидни имейли:

Регулярен израз за имейл адреси

```
email_regex = ~r/[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/
```

```
text = ""
```

Примерен текст с имейли:

```
test@example.com,
```

```
hello.world@domain.co,
```

```
invalid.email@.com,
```

```
another.test@sub.domain.com
```

```
""
```

Намиране на валидни имейли

```
valid_emails = Regex.scan(email_regex, text)
```

```
IO.inspect(valid_emails)
```

Изход

Този код ще открие и изведе списък с валидни имейл адреси:

```
[  
  ["test@example.com"],  
  ["hello.world@domain.co"],  
  ["another.test@sub.domain.com"]  
]
```

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

Регулярният израз `~r/[a-zA-Z0-9._%+~]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}/` е предназначен за валидиране на имейл адреси.

Основна структура на регулярен израз

- **Обозначаване на регулярния израз:**

- ~r/.../ е синтаксисът в Elixir за създаване на регулярни изрази. Всички използвани символи между / се считат за част от регулярния израз.
- **[a-zA-Z0-9._%+~]**+:
 - ❖ **[a-zA-Z0-9._%+~]** указва с какви символи може да започне имейл адресът:
 - a-z: малки букви
 - A-Z: големи букви
 - 0-9: цифри
 - .: точка
 - _: долна черта
 - %: символ % (обикновено се използва в имейл адреси)
 - +: символ + (използва се често в имейл адреси)
 - -: символ - (използва се често в имейл адреси)
- **+** след затварящата квадратна скоба указва, че предходната група символи трябва да се появи поне веднъж. Това означава, че името на имейл адреса не може да бъде празно.

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

➤ @:

Символът @ е фиксирана част от имейл адреса, която отделя местна част (username) от домейн.

➤ [a-zA-Z0-9.-]+:

Подобно на предишната част, но тук определя домейн частта на имейл адреса:

- a-z: малки букви
- A-Z: големи букви
- 0-9: цифри
- .: точка
- -: тире

❖ + указва, че тази част трябва да се появи поне веднъж, отражаваща изискванията за валидни домейни.

➤ \.:

Символът \. обозначава буквална точка. Знакът . в регулярни изрази обикновено съвпада с всеки символ, затова преди него се слага обратна наклонена черта (\), за да укажем, че искаме да съвпаднем с действителната точка.

➤ [a-zA-Z]{2,}:

Последната част указва разширението на домейна:

- [a-zA-Z] указва, че разширението трябва да съдържа само букви.
- {2,} указва, че името на домейна (например, .com, .org, .bg) трябва да е поне с две букви, но не определя максимален брой.

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

Следователно, този регулярния израз служи за валидиране на имейл адреси, чиято структура е следната:

- Започва с комбинация от букви, цифри, точки, долни черти, проценти, плюсове и тирета.
- Следва символ @.
- След него идва домейн, който може да включва букви, цифри, точки и тирета.
- Завършва с точка, последвана от разширение, съдържащо поне две букви.

Ограничения

Обаче, имайте предвид, че този регулярният израз не покрива всички възможни валидни имейл адреси, поради сложността на спецификацията на имейл адресите. Това е основен шаблон и не улавя всички изключения и вариации в имейл адресите. За сложна валидизация на имейли, обикновено е необходима по-подробна логика.

Обобщение

Работата с регулярни изрази в Elixir е мощен инструмент за търсене и манипулиране на текст. Модулът Regex предлага всички необходими функции за търсене, извличане и замяна на текстови модели.

Тема 6. Контролиране на пътя на изпълнение на програмата с pattern matching. Принципи и правила за прилагане към различните типове данни. Приложение при дефинирането на функции

Използвана литература:

- [1] Chris McCord, “Metaprogramming Elixir”, 2015, ISBN-13: 978-1-68050-041-7
- [2] SAŠA JURIC, “Elixir in Action”, Second Edition, May 2015, ISBN 9781617292019
- [3] John Paul Mueller “Functional Programming For Dummies”, 2019, ISBN 978-1-119-52751-0 (ebk); ISBN ePDF 978-1-119-52749-7 (ebk)
- [4] Примери: <https://elixir-examples.github.io/single-page>
- [5] Примери: <https://elixirschool.com/en/lessons/basics/functions/>
- [6] <https://elixir-lang.readthedocs.io/en/latest/intro/4.html>
- [7] Примери: <https://elixir-lang.org/crash-course.html#control-flow>